

Project Interim Report
On
Vehicle Load Monitoring

(Using Python, ML (Sk-learn), OpenCV, libraries, accelerometer, camera, Jupyter.)

RU-500-02-00002
Foundation of Machine Learning

SESSION: 2025-26



Submitted By
Soumya Soni
RU-25-11127

**Master of Computer Application with AI and ML in
association with Google**

Under the Guidance of
Mr. Zamin Hamid

SCHOOL OF INFORMATION TECHNOLOGY
RUNGTA INTERNATIONAL SKILLS UNIVERSITY
BHILAI, CG

Abstract

This project presents an intelligent and automated system for detecting overloaded freight vehicles on highways using a combination of vibration sensor technology and computer vision techniques. The system is designed to improve road safety, reduce infrastructure damage, and support efficient transportation management by identifying overloaded vehicles in real time.

Overloaded freight vehicles are one of the major causes of road accidents, excessive wear and tear on highways, increased fuel consumption, and reduced vehicle lifespan. Traditional methods of detecting overloaded vehicles, such as manual inspections and weighbridge checkpoints, are often time-consuming, labor-intensive, and prone to human error. These limitations create a strong need for a smart, automated, and reliable monitoring solution capable of operating continuously without direct human intervention.

The proposed system utilizes vibration sensors, specifically accelerometers, to capture and analyze vibration patterns generated when vehicles pass through a monitoring zone. Since overloaded vehicles exert greater force on the road surface, they produce distinct vibration signatures compared to normally loaded vehicles. The collected sensor data is processed using machine learning algorithms to classify vehicles as either safe or overloaded based on learned patterns.

To enhance detection accuracy, a camera-based visual monitoring module is integrated into the system. The camera captures images or video frames of passing vehicles, enabling vehicle identification and providing additional evidence for overload detection. Image processing and computer vision techniques can be used to extract vehicle-related information such as vehicle type, dimensions, and registration details. The fusion of vibration sensor data and visual information significantly improves the reliability and robustness of the detection process.

When an overloaded vehicle is detected, the system automatically generates real-time alerts for highway authorities or monitoring personnel. The vehicle information, detection timestamp, and overload status are recorded in a centralized database for future analysis, reporting, and enforcement purposes. This automated reporting mechanism helps authorities take immediate action while maintaining accurate records of violations.

The system offers several advantages, including high detection accuracy, real-time monitoring capability, reduced dependence on manual inspections, lower operational costs, and continuous 24/7 surveillance. It can be deployed at highway toll plazas, checkpoints, industrial transportation corridors, logistics hubs, and smart city traffic management systems.

By combining Internet of Things (IoT) sensors, machine learning, and computer vision technologies, the proposed solution provides a modern and scalable approach to freight vehicle monitoring. The implementation of this system can contribute to safer roads, longer-lasting transportation infrastructure, improved traffic management, and enhanced compliance with vehicle load regulations. Overall, the Smart Overloaded Freight Vehicle Detection System represents an effective and innovative solution for addressing the growing challenges associated with overloaded freight transportation in modern highway networks.

Introduction

The Smart Vehicle Load Monitoring System is an intelligent solution designed to detect overloaded freight vehicles on highways using vibration sensors and visual detection techniques. The system monitors vibration patterns generated by moving vehicles and analyzes them to estimate load conditions. A camera module is also integrated to capture vehicle information, providing additional support for accurate detection and monitoring.

Overloaded vehicles are one of the major causes of road accidents, increased fuel consumption, reduced vehicle lifespan, and severe damage to road infrastructure. Traditional load inspection methods, such as manual checking and weighbridge stations, are often time-consuming, expensive, and inefficient for continuous highway monitoring. These limitations highlight the need for an automated and reliable vehicle load detection system.

The proposed system continuously collects vibration data from sensors installed near the roadway. Since overloaded vehicles produce stronger vibration signatures than normally loaded vehicles, the system can identify abnormal load conditions through data analysis. The visual detection module captures images of passing vehicles, helping to verify and record vehicle details.

When an overloaded vehicle is detected, the system generates real-time alerts and stores the vehicle information for further analysis and enforcement actions. This automated approach reduces human effort, improves monitoring efficiency, and enables faster response by highway authorities.

The system offers several advantages, including real-time operation, improved detection accuracy, reduced dependency on manual inspections, and enhanced road safety. It can be deployed at highways, toll plazas, transport checkpoints, and logistics corridors to ensure compliance with load regulations. Overall, the Smart Vehicle Load Monitoring System provides a reliable, cost-effective, and efficient solution for modern highway monitoring and safer transportation management.

Scope

This project is designed for monitoring overloaded freight vehicles on highways using vibration sensors and visual detection techniques. It provides an efficient and cost-effective solution for identifying vehicles that exceed permissible load limits, helping to improve road safety and reduce damage to highway infrastructure. The system is particularly suitable for small-scale highway monitoring setups and can be deployed at toll plazas, checkpoints, industrial transport routes, and entry/exit points where basic vehicle load monitoring is required.

The proposed system analyzes vibration patterns generated by passing vehicles and uses visual detection to support the identification process. By combining these technologies, the system can automatically detect potential overload conditions and generate alerts without requiring continuous manual supervision. This helps reduce inspection time, improve monitoring efficiency, and ensure better compliance with transportation regulations.

The project can be further enhanced by integrating advanced database management systems to store vehicle information, load history, violation records, timestamps, and monitoring reports. Such a database would allow authorities to track repeat offenders, generate statistical reports, and maintain a centralized record of vehicle activities for future analysis and enforcement.

In future developments, the system can be expanded with Artificial Intelligence (AI) and Machine Learning (ML) techniques to improve detection accuracy and automate vehicle classification. Advanced image processing algorithms can be used to identify vehicle types, recognize license plates, and estimate load conditions more accurately from captured images and video streams.

Cloud connectivity can also be incorporated to enable secure storage and real-time access to monitoring data from multiple locations. This would allow traffic authorities and highway management agencies to monitor vehicle activity remotely through web dashboards and mobile applications. Real-time data sharing, automated notifications, and centralized monitoring can significantly improve decision-making and enforcement processes.

Additional features such as GPS tracking integration, predictive analytics, automated report generation, and IoT-based sensor networks can further enhance the system's capabilities. With these improvements, the project can evolve into a comprehensive smart highway monitoring solution capable of supporting large-scale transportation management, improving road safety, reducing infrastructure maintenance costs, and promoting efficient freight vehicle operations.

System Requirements

Hardware Requirements

- Computer/Laptop with **4 GB RAM or higher** (8 GB recommended)
- **Dual-core Processor** or above (Intel Core i3/i5, AMD Ryzen 3 or equivalent)
- Minimum **20 GB free storage space**
- Webcam or external camera (for visual vehicle detection)
- Accelerometer/Vibration Sensor (for vibration data collection)
- Stable internet connection (for updates and cloud integration, if required)
- Display monitor with minimum resolution of **1366 × 768**

Software Requirements

- **Operating System:** Windows 10/11, Linux, or macOS
- **Python 3.x**
- **Jupyter Notebook** or **Visual Studio Code**
- **Anaconda Distribution** (optional)
- **Git** (optional for version control)

Python Libraries

- **NumPy** – Numerical computations
- **Pandas** – Data manipulation and analysis
- **Matplotlib** – Data visualization
- **Seaborn** – Statistical data visualization
- **Scikit-Learn** – Machine learning algorithms
- **OpenCV** – Image and video processing
- **TensorFlow/Keras** (optional for advanced AI models)
- **Joblib** – Model saving and loading
- **SciPy** – Scientific computing
- **Pillow (PIL)** – Image processing
- **Streamlit** (optional) – Web application interface

Database Requirements (Optional)

- **MySQL** or **SQLite** for storing vehicle records, load history, and violation details

Development Tools

- Jupyter Notebook
- Visual Studio Code
- Anaconda Navigator (optional)
- GitHub (for project repository and version control)

These requirements are sufficient for developing, training, and deploying the **Smart Vehicle Load Monitoring System using Vibration Sensors and Visual Detection Techniques**.

Methodology

- **System Setup**

- Install vibration sensors and cameras at highways, toll plazas, or checkpoints.
- Connect all devices to a central processing unit for data collection and analysis.

- **Vehicle Detection**

- The system automatically detects a vehicle when it enters the monitoring area.
- Sensors and cameras are activated to start the monitoring process.

- **Vibration Data Collection**

- Vibration sensors capture vibration patterns generated by the moving vehicle.
- The intensity of vibrations depends on the vehicle's weight and load condition.

- **Image Acquisition**

- The camera captures images or video frames of the passing vehicle.
- Visual data helps identify and verify the monitored vehicle.

- **Data Preprocessing**

- The collected vibration signals are filtered to remove noise and unwanted disturbances.
- The processed data is prepared for accurate analysis.

- **Feature Extraction**

- Important features such as vibration amplitude, frequency, and signal characteristics are extracted.
- These features are used to determine the vehicle's load condition.

- **Load Analysis**

- The extracted data is analyzed using predefined thresholds or machine learning algorithms.
- The system estimates whether the vehicle is operating within the permissible load limit.

- **Overload Detection**

- The estimated load is compared with the standard allowable load limit.
- Vehicles exceeding the limit are classified as overloaded.

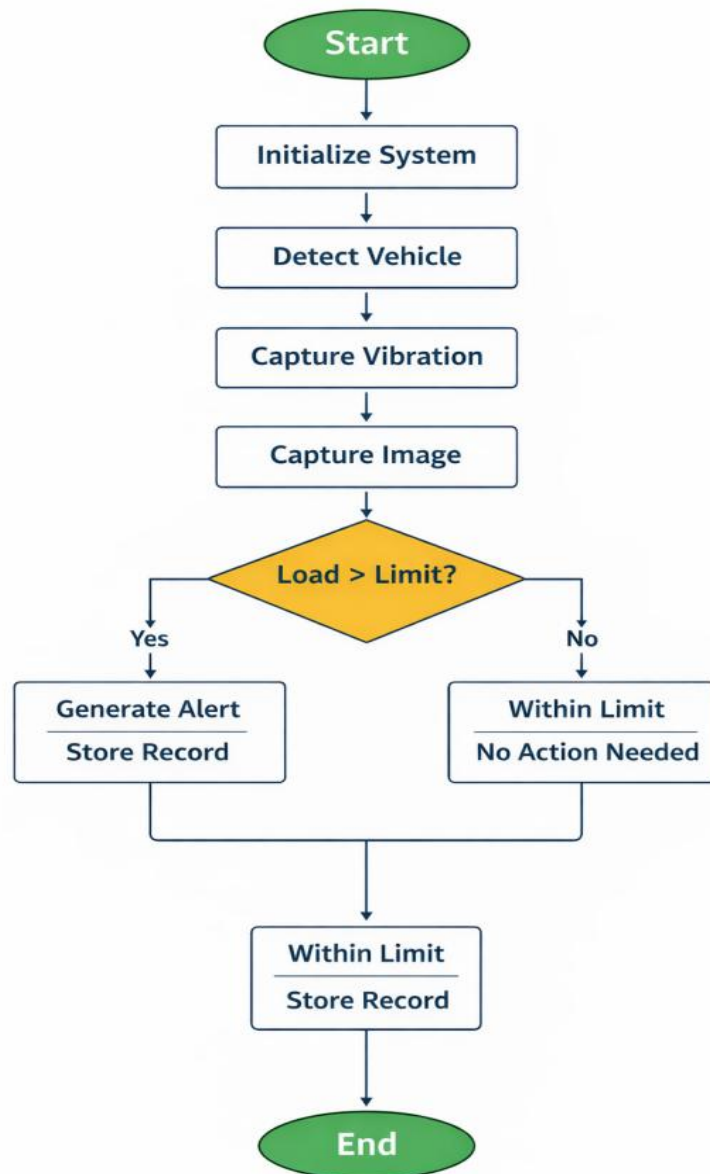
- **Alert Generation**

- When an overloaded vehicle is detected, the system generates a real-time alert.
- The alert notifies monitoring personnel for further action.

- **Data Storage and Reporting**

- Vehicle details, load status, captured images, and detection time are stored in a database.

Flowchart



Variables inside class:

- `vehicleID` – stores vehicle identification number
- `vibrationLevel` – stores vibration sensor readings
- `loadLimit` – maximum allowed load value
- `isOverloaded` – status of vehicle (true/false)
- `imageData` – stores captured image data

Methods inside class:

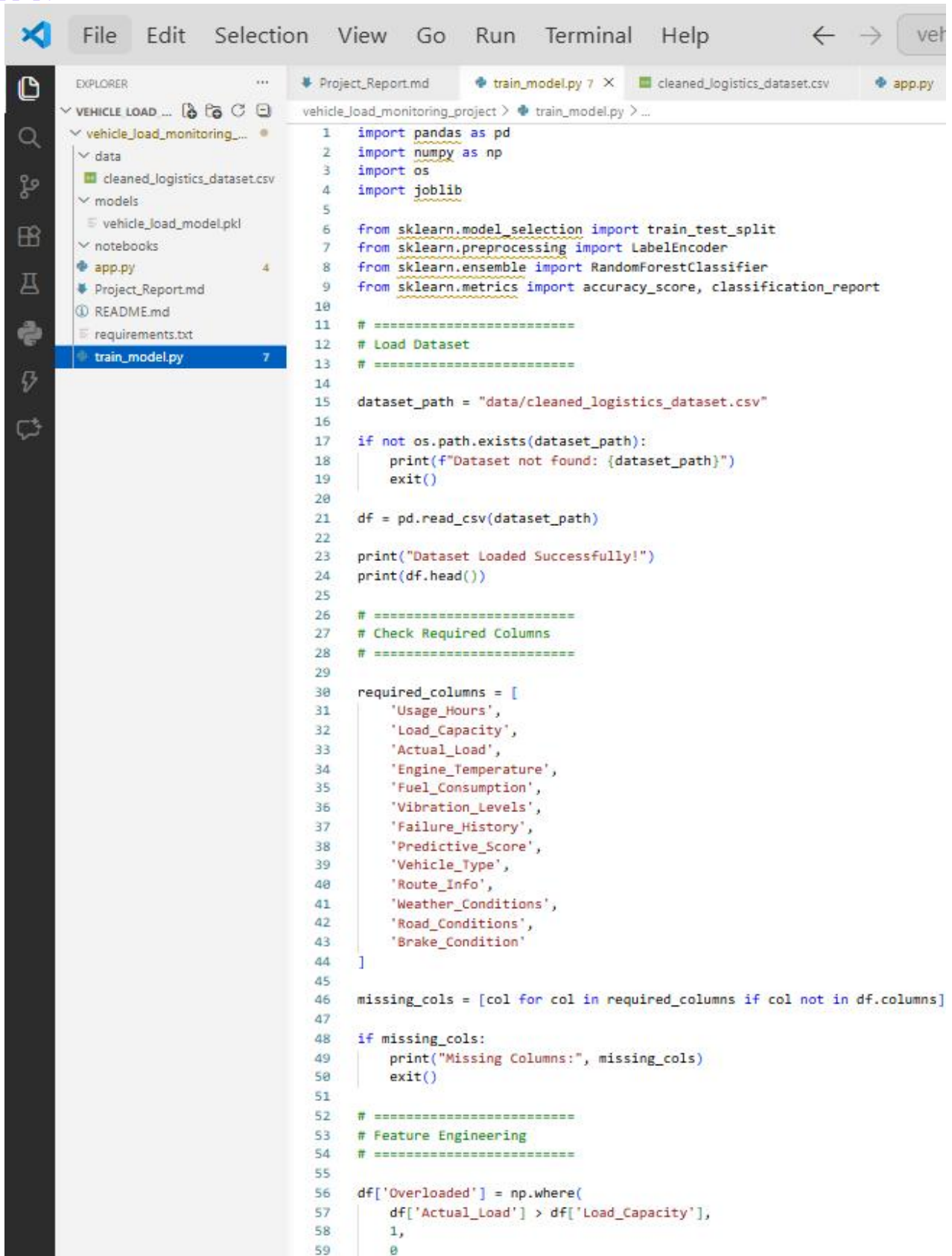
- `captureVibration()` – reads vibration sensor data
- `captureImage()` – captures vehicle image
- `analyzeLoad()` – processes data to estimate load
- `checkOverload()` – compares load with limit
- `generateAlert()` – sends alert if overloaded

Array (if used):

- `vehicleRecords[]` – stores details of multiple vehicles detected

Vehicle load Monitoring

App.py



The screenshot shows a Visual Studio Code editor window with a project named 'vehicle_load_monitoring_project'. The Explorer sidebar on the left shows the project structure, including a 'data' folder with 'cleaned_logistics_dataset.csv', a 'models' folder with 'vehicle_load_model.pkl', and a 'notebooks' folder. The 'train_model.py' file is selected and open in the editor. The code in the editor is as follows:

```
1 import pandas as pd
2 import numpy as np
3 import os
4 import joblib
5
6 from sklearn.model_selection import train_test_split
7 from sklearn.preprocessing import LabelEncoder
8 from sklearn.ensemble import RandomForestClassifier
9 from sklearn.metrics import accuracy_score, classification_report
10
11 # =====
12 # Load Dataset
13 # =====
14
15 dataset_path = "data/cleaned_logistics_dataset.csv"
16
17 if not os.path.exists(dataset_path):
18     print(f"Dataset not found: {dataset_path}")
19     exit()
20
21 df = pd.read_csv(dataset_path)
22
23 print("Dataset Loaded Successfully!")
24 print(df.head())
25
26 # =====
27 # Check Required Columns
28 # =====
29
30 required_columns = [
31     'Usage_Hours',
32     'Load_Capacity',
33     'Actual_Load',
34     'Engine_Temperature',
35     'Fuel_Consumption',
36     'Vibration_Levels',
37     'Failure_History',
38     'Predictive_Score',
39     'Vehicle_Type',
40     'Route_Info',
41     'Weather_Conditions',
42     'Road_Conditions',
43     'Brake_Condition'
44 ]
45
46 missing_cols = [col for col in required_columns if col not in df.columns]
47
48 if missing_cols:
49     print("Missing Columns:", missing_cols)
50     exit()
51
52 # =====
53 # Feature Engineering
54 # =====
55
56 df['Overloaded'] = np.where(
57     df['Actual_Load'] > df['Load_Capacity'],
58     1,
59     0
```


Train_model.py

File Edit Selection View Go Run Terminal Help

vehicle_load_monitoring_project

Project_Report.md X README.md train_model.py 7 X cleaned_logistics_c

EXPLORER

VEHICLE_LOAD_...

vehicle_load_monitoring_...

data

cleaned_logistics_dataset.csv

models

vehicle_load_model.pkl

notebooks

app.py 4

Project_Report.md

README.md

requirements.txt

train_model.py 7

C:\Users\Soumya

Soni\Downloads\vehicle_load_monitoring_project\vehicle_load_monitoring_project\Project_Report.md

```
3 import os
4 import joblib
5
6 from sklearn.model_selection import train_test_split
7 from sklearn.preprocessing import LabelEncoder
8 from sklearn.ensemble import RandomForestClassifier
9 from sklearn.metrics import accuracy_score, classification_report
10
11 # =====
12 # Load Dataset
13 # =====
14
15 dataset_path = "data/cleaned_logistics_dataset.csv"
16
17 if not os.path.exists(dataset_path):
18     print(f"Dataset not found: {dataset_path}")
19     exit()
20
21 df = pd.read_csv(dataset_path)
22
23 print("Dataset Loaded Successfully!")
24 print(df.head())
25
26 # =====
27 # Check Required Columns
28 # =====
29
30 required_columns = [
31     'Usage_Hours',
32     'Load_Capacity',
33     'Actual_Load',
34     'Engine_Temperature',
35     'Fuel_Consumption',
36     'Vibration_Levels',
37     'Failure_History',
38     'Predictive_Score',
39     'Vehicle_Type',
40     'Route_Info',
41     'Weather_Conditions',
```

PROBLEMS 11 OUTPUT DEBUG CONSOLE TERMINAL PORTS

json-server is now active!

0 11 BLACKBOX Agent Open Website

FileEditSelectionViewGoRunTerminalHelp

vehicle_load_monitoring_project

EXPLORER

VEHICLE_LOAD_...
vehicle_load_monitoring_...
data
cleaned_logistics_dataset.csv
models
vehicle_load_model.pkl
notebooks
app.py
Project_Report.md
README.md
requirements.txt
train_model.py

Project_Report.mdREADME.mdtrain_model.py 7 Xcleaned_logistics_dataset.csv

vehicle_load_monitoring_project > train_model.py > ...
39 'Vehicle_Type',
40 'Route_Info',
41 'Weather_Conditions',
42 'Road_Conditions',
43 'Brake_Condition'
44]
45
46 missing_cols = [col for col in required_columns if col not in df.columns]
47
48 if missing_cols:
49 print("Missing Columns:", missing_cols)
50 exit()
51
52 # =====
53 # Feature Engineering
54 # =====
55
56 df['Overloaded'] = np.where(
57 df['Actual_Load'] > df['Load_Capacity'],
58 1,
59 0
60)
61
62 # =====
63 # Encode Categorical Columns
64 # =====
65
66 categorical_cols = [
67 'Vehicle_Type',
68 'Route_Info',
69 'Weather_Conditions',
70 'Road_Conditions',
71 'Brake_Condition'
72]
73
74 encoders = {}
75
76 for col in categorical_cols:
77 le = LabelEncoder()
78 df[col] = le.fit_transform(df[col].astype(str))
79 encoders[col] = le

PROBLEMS 11OUTPUTDEBUG CONSOLETERMINALPORTS
json-server is now active!

< 0 11 BLACKBOX Agent Open Website

VS Code interface showing a Python script for vehicle load monitoring model training.

EXPLORER

- VEHICLE LOAD ...
 - data
 - cleaned_logistics_dataset.csv
 - models
 - vehicle_load_model.pkl
 - notebooks
 - app.py
 - Project_Report.md
 - README.md
 - requirements.txt
 - train_model.py**

train_model.py

```
81 # =====
82 # Features and Target
83 # =====
84
85 features = required_columns
86 target = 'Overloaded'
87
88 X = df[features]
89 y = df[target]
90
91 # =====
92 # Split Data
93 # =====
94
95 X_train, X_test, y_train, y_test = train_test_split(
96     X,
97     y,
98     test_size=0.2,
99     random_state=42
100 )
101
102 # =====
103 # Train Model
104 # =====
105
106 model = RandomForestClassifier(
107     n_estimators=100,
108     random_state=42
109 )
110
111 model.fit(X_train, y_train)
112
113 # =====
114 # Predictions
115 # =====
116
117 preds = model.predict(X_test)
118
119 # =====
120 # Evaluation
121 # =====
122
123 accuracy = accuracy_score(y_test, preds)
124
125 print("\n=====")
126 print("Model Accuracy:", accuracy)
127 print("=====\n")
128
129 print("Classification Report:\n")
130 print(classification_report(y_test, preds))
131
132 # =====
133 # Save Model
134 # =====
135
136 os.makedirs("models", exist_ok=True)
137
138 model_path = "models/vehicle_load_model.pkl"
139
140 joblib.dump(model, model_path)
141
142 print(f"\nModel saved successfully at: {model_path}")
```

PROBLEMS 11 **OUTPUT** **DEBUG CONSOLE** **TERMINAL** **PORTS**

json-server is now active!

0 11 BLACKBOX Agent Open Website

Implementation

The implementation of the **Smart Vehicle Load Monitoring System** follows a modular and object-oriented approach to ensure efficient operation, maintainability, and scalability. The system is divided into different classes, with each class performing a specific function in the vehicle monitoring process.

The **VehicleMonitor** class acts as the central controller of the system. It coordinates communication between all modules, manages the workflow, and controls the sequence of operations from vehicle detection to alert generation. This class ensures that all components work together smoothly and efficiently.

The **Sensor** class is responsible for collecting vibration data generated when a vehicle passes through the monitoring zone. It continuously reads vibration signals from the accelerometer and sends the collected data to the processing module for analysis. The vibration patterns provide valuable information about the vehicle's load condition.

The **Camera** class captures images or video frames of passing vehicles. These images help identify the vehicle and provide visual evidence for overload detection. The visual data can also be stored for future verification and reporting purposes.

After data collection, the **Data Processing Module** filters and preprocesses the vibration signals to remove unwanted noise and improve data quality. Relevant features are extracted from the processed data and used for load estimation. The system then analyzes the vibration characteristics and determines whether the vehicle is operating within the permissible load range.

The estimated load is compared with predefined threshold values stored in the system. If the load remains below the specified limit, the vehicle is classified as safe. If the load exceeds the allowable limit, the system identifies the vehicle as overloaded.

The **AlertSystem** class automatically generates a warning message whenever an overloaded vehicle is detected. The alert can be displayed on a monitoring dashboard or sent to the concerned authority for immediate action. At the same time, the vehicle image, load status, detection time, and alert details are recorded in the database.

The system also maintains a history of detected vehicles and overload violations, allowing authorities to review records and generate reports when needed. This automated implementation reduces manual effort, increases monitoring accuracy, and enables real-time detection of overloaded freight vehicles.

Process Flow:

Vehicle Detection → Vibration Data Collection → Image Capture → Data Processing → Load Analysis → Threshold Comparison → Overload Detection → Alert Generation → Data Storage and Reporting.

Gantt Chart

A Gantt chart helps in organizing the project into clear and manageable tasks such as requirement analysis, system design, data collection, coding, model development, testing, documentation, and deployment. It provides a structured timeline for each phase of the project and ensures that all activities are completed in a logical sequence.

The Gantt chart allows project developers to track progress, monitor deadlines, and allocate resources effectively throughout the development process. By dividing the project into smaller tasks with specific start and end dates, it becomes easier to manage workload and identify dependencies between different activities.

During the development of the Smart Vehicle Load Monitoring System, the Gantt chart can be used to schedule important stages such as sensor integration, camera setup, software development, machine learning model implementation, system testing, and final evaluation. This helps ensure that each component is completed on time and integrated successfully with the overall system.

The chart also assists in identifying potential delays and risks at an early stage, allowing corrective actions to be taken before they affect the project timeline. Team members can clearly understand their responsibilities and coordinate their work more efficiently.

Even when rebuilding, upgrading, or modifying the project in the future, proper planning through a Gantt chart ensures smooth execution, better coordination, and systematic development. It improves project management, enhances productivity, and increases the likelihood of completing the project within the planned schedule and budget. Overall, a Gantt chart serves as an effective tool for organizing, monitoring, and controlling every stage of the project

Task	Week 1	Week 2	Week 4
Project Planning			
System Design			
Data Collection Setup			
Coding & Development			
Testing & Optimization			
Final Implementation			

Conclusion

The project successfully demonstrates the application of **Machine Learning and Python-based technologies** to develop an intelligent and efficient vehicle load monitoring system. By combining vibration sensor data with visual detection techniques, the system provides a reliable method for identifying overloaded freight vehicles in real time.

The proposed solution helps address common problems associated with vehicle overloading, such as road accidents, infrastructure damage, increased maintenance costs, and reduced vehicle performance. Through automated monitoring and analysis, the system minimizes the need for manual inspections and improves the efficiency of highway monitoring operations.

The implementation of vibration-based load estimation allows the system to detect abnormal load conditions accurately, while the camera module provides additional verification and visual records of monitored vehicles. The integration of these technologies enhances overall detection accuracy and reliability.

The system is capable of generating alerts whenever an overloaded vehicle is detected, enabling authorities to take timely action. In addition, vehicle information and monitoring records can be stored for future analysis, reporting, and regulatory purposes.

The project demonstrates how machine learning, sensor technology, and computer vision can work together to create a practical and cost-effective solution for transportation management. It offers real-time monitoring capabilities, scalability, and the potential for integration with modern smart traffic systems.

Overall, the Smart Vehicle Load Monitoring System provides an effective approach to improving road safety, protecting highway infrastructure, and supporting efficient freight transportation. Future enhancements such as advanced AI models, cloud-based monitoring, IoT integration, and automated reporting can further increase the system's performance and applicability in large-scale smart highway management environments.

References

- [1] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.
- [2] S. Raschka and V. Mirjalili, *Python Machine Learning: Machine Learning and Deep Learning with Python, Scikit-Learn, and TensorFlow*, 3rd ed. Birmingham, U.K.: Packt Publishing, 2019.
- [3] R. B. Machemehl and M. Wang, “Effects of overloaded trucks on highway pavement performance,” *Journal of Transportation Engineering*, vol. 128, no. 6, pp. 511–518, Nov. 2002.
- [4] S. S. Rao, *Mechanical Vibrations*, 6th ed. Upper Saddle River, NJ, USA: Pearson Education, 2017.
- [5] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY, USA: Pearson, 2018.
- [6] GeeksforGeeks, “Machine Learning and Python Tutorials.” [Online]. Available: <https://www.geeksforgeeks.org>. [Accessed: Aug. 05, 2026].
- [7] Kaggle, “Machine Learning Datasets and Competitions.” [Online]. Available: <https://www.kaggle.com>. [Accessed: Aug. 05, 2026].
- [8] OpenCV, “Open Source Computer Vision Library.” [Online]. Available: <https://opencv.org>. [Accessed: Aug. 05, 2026].
- [9] Python Software Foundation, “Python 3.x Documentation.” [Online]. Available: <https://docs.python.org/3>. [Accessed: Aug. 05, 2026].
- [10] NumPy Developers, “NumPy Documentation.” [Online]. Available: <https://numpy.org/doc>. [Accessed: Aug. 05, 2026].
- [11] Pandas Development Team, “Pandas Documentation.” [Online]. Available: <https://pandas.pydata.org/docs>. [Accessed: Aug. 05, 2026].
- [12] Matplotlib Development Team, “Matplotlib Documentation.” [Online]. Available: <https://matplotlib.org/stable>. [Accessed: Aug. 05, 2026].
- [13] Scikit-Learn Developers, “Scikit-Learn Documentation.” [Online]. Available: <https://scikit-learn.org/stable>. [Accessed: Aug. 05, 2026].